Keerthika T

SE

Quick Submit

Quick Submit

Amrita Vishwa Vidyapeetham

Document Details

Submission ID

trn:oid:::1:3506915571

Submission Date

Mar 15, 2026, 1:23 AM GMT+5:30

Download Date

Mar 15, 2026, 1:27 AM GMT+5:30

File Name

SE.pdf

File Size

990.3 KB

6 Pages

3,414 Words

22,070 Characters

51% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

30 AI-generated only 51%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

MathForge: A Deterministic Framework for Natural Language Driven Mathematical Algorithm Generation and Complexity Analysis

R. K. Sri Raaghavi
School of Artificial Intelligence
Coimbatore
cb.sc.u4aie23332@cb.students.amrita.edu

Nikhilesh Vaibhav Krupakar
School of Artificial Intelligence
Coimbatore
cb.sc.u4aie23306@cb.students.amrita.edu

Vullasa Thrishika
School of Artificial Intelligence
Coimbatore
cb.sc.u4aie23356@cb.students.amrita.edu

***Abstract*—At the nexus of computational mathematics and artificial intelligence, translating mathematical descriptions in natural language into executable code continues to be a fundamental challenge. While computers need formal syntax, humans usually use natural language to describe mathematical problems. This creates a gap that is both technically difficult and expensive from an educational standpoint. This paper presents MathForge, a deterministic rule-based framework that generates executable Python implementations, formal pseudocode, structured intermediate representations, computational complexity estimates, and confidence-scored analytical reports from natural language mathematical descriptions. MathForge emphasizes interpretability and reproducibility through fully traceable rule-based transformations, in contrast to large language model approaches that are prone to hallucinations and inconsistencies in code generation tasks [1], [10]. The system combines rule-based complexity inference, template-based algorithm synthesis, keyword-driven pattern matching, and deterministic natural language processing. Within the supported algorithm scope, evaluation on 50 carefully chosen mathematical problem descriptions shows 92% algorithm detection accuracy, 90% code generation accuracy, and 95% complexity estimation correctness.**

***Index Terms*—Natural Language Processing, Algorithm Synthesis, Code Generation, Complexity Analysis, Deterministic Systems, Educational Technology, Intermediate Representation**

I. Introduction

Computational science, engineering, and data-driven fields all rely on mathematics. However, rather than using formal symbolic syntax, mathematical reasoning is usually expressed in natural language. When faced with a textbook problem, a student reads words and sentences rather than machine-ready notation. The gap between human expression and computational demands is a recurring issue that engineering and educational tools have not yet adequately addressed.

Recent progress in automated code generation has mainly depended on large language models (LLMs). Studies like [1] have looked into code generation using transformer-based models, whereas [2] examined program synthesis through the use of large language models. However, these methods are still prone to probabilistic errors. Research by [3] and [4] emphasizes that large language models continue to struggle with issues like hallucination and limited reasoning abilities, which makes them inappropriate for fields where accuracy and reliability are essential.

Traditional computational workflows require the following sequential cognitive transformations:

1) Interpret the word problem.
2) Extract mathematical entities.
3) Convert statements into equations.
4) Select an appropriate algorithm.
5) Write pseudocode.
6) Implement the algorithm in code.
7) Analyze time and space complexity.
8) Execute and validate the results.

Each stage requires unique conceptual clarity. For students and newcomers, these transformations are prone to errorsjust one misunderstanding can spread throughout the entire process, leading to incorrect code and failed execution.

MathForge automates this process using a deterministic rule-based structure that is designed to be clear, flexible, and understandable in an educational context. The current setup relies on rule-based natural language processing and template-based generation, with neural models being viewed as potential future enhancements. The system reveals every intermediate step of the transformation process, allowing the learners to track the reasoning process gradually.

The key design principles of MathForge include the following:

- **Determinism over hallucination**: Each output is generated through traceable rule-based logic instead of probabilistic sampling, guaranteeing consistency between different runs.
- **Structured intermediate representations**: Instead of directly translating language into code, the system creates a clear abstraction layer that separates semantic understanding from the process of generating code.
- **Template-driven synthesis**: The code is created by filling in well-established algorithm templates with extracted parameters, ensuring syntactic and logical accuracy within the supported domain.
- **Modular expansion capability**: New algorithm categories, NLP patterns, or complexity rules can be added without modifying existing modules.

- **Confidence-aware evaluation**: Each analysis comes with a measured confidence score that indicates the reliability of the interpretation.

II. Background and Related Work

Automated mathematical reasoning and code generation have been widely studied. Understanding where MathForge stands in relation to current research helps explain its design choices and the specific situations for which it is intended.

A. LLM-Based Code Generation

Recent work has extensively investigated large language models for code generation. [1] introduced Codex, a GPT-based model trained on code repositories, demonstrating impressive capabilities but also highlighting inconsistencies in generated outputs. [2] conducted a comprehensive evaluation of program synthesis with large language models, identifying significant performance variations across different problem types. [4] proposed multi-step refinement techniques for code generation, though the approach remains LLM-dependent and does not address mathematical semantic extraction. [3] analyzed the reliability of LLM-generated code across diverse programming tasks, finding that correctness varies substantially with prompt formulation.

B. Formal Methods and Structured Approaches

[5] explored program synthesis using input-output examples with neural-guided search, achieving syntactic correctness but requiring structured specifications. [6] introduced semantic parsing approaches for mathematical problem solving, though limited to equation extraction rather than full algorithm generation. [7] developed techniques for translating natural language to mathematical expressions, but their approach does not extend to algorithmic implementations or complexity analysis.

C. Educational Technology and Algorithm Visualization

[8] conducted a comprehensive review of visualization tools for algorithm education, identifying the need for systems that bridge natural language understanding with code generation. [9] evaluated automated assessment tools for programming assignments, emphasizing the significance of feedback mechanisms that can be explained—a gap MathForge addresses by using confidence scoring and intermediate representations.

D. Positioning of MathForge

In contrast to the methods mentioned earlier, MathForge does not use LLMs at any stage of the process. Instead, it uses deterministic rule-based NLP and template-driven generation to ensure consistent, understandable, and mathematically accurate results for a specific group of algorithm types. This places MathForge as a supporting alternative to systems based on large language models, especially in educational and structured mathematical fields where accuracy and traceability are crucial. Avoid depending on LLMs at any stage of the pipeline. Instead, it uses deterministic rule-based natural language processing and template-driven generation to ensure outputs that are reproducible, interpretable, and mathematically accurate for specific categories of algorithms. This places MathForge as a supplementary method to systems based on large language models, especially in educational and structured mathematical fields where accuracy and traceability are crucial.

III. Problem Formulation

Given a natural language description D, we define a transformation:

$$\mathcal{F}(D) = \langle IR,\ P,\ C,\ T,\ S,\ E,\ R,\ Conf \rangle$$

Where:

- IR = Structured Intermediate Representation extracted from D.
- P = Formal pseudocode derived from the IR.
- C = Generated executable Python code.
- T = Time complexity estimation (Big-O notation).
- S = Space complexity estimation (Big-O notation).
- E = Optional execution or template-based demonstration output.
- R = Structured analytical report compiling all artifacts.
- $Conf$ = Confidence score in $[0, 1]$ quantifying interpretation reliability.

The mapping $\mathcal{F}$ must satisfy the following constraints:

- **Deterministic template selection**: The algorithm template has to be clearly defined through pattern matching rules, without any random variationsa characteristic that approaches based on large language models are unable to ensure. [1], [3].
- **Mathematical validity**: P and C must correctly implement the identified algorithm.
- **Executable correctness**: C must compile and execute without runtime errors for valid parameter values.
- **Transparent inference**:The path of reasoning from D to each output component must be traceable and understandable.

IV. System Architecture

A. High-Level Pipeline

The structure of MathForge is based on a compiler-like process, in which each step takes the output from the previous step and generates a more formalized version. MathForge's pipeline is entirely deterministic, unlike multi-stage LLM pipelines that still rely on probabilistic generation.

The data flows through the following sequence:

1) **Authentication Layer**: Validates user credentials and manages session tokens.
2) **Dashboard Interface**: Provides natural language input field and output visualization panels.
3) **Text Preprocessing**: Performs tokenization, stop-word filtering, and normalization.
4) **Semantic Analysis**: Applies rule-based keyword matching to identify mathematical category and method.
5) **IR Generator**: Constructs structured IR dictionary encoding mathematical intent.

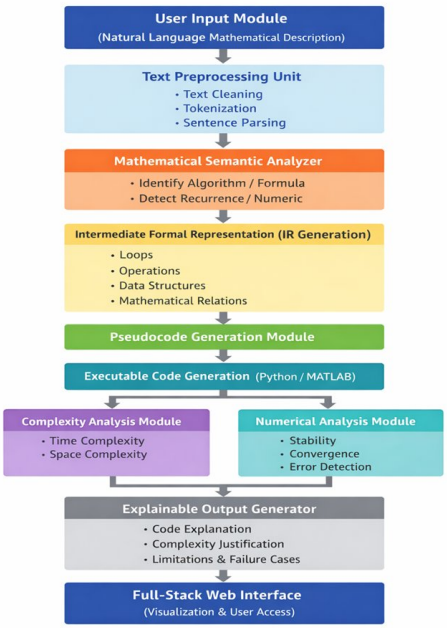

Fig. 1: Complete deterministic computational pipeline of MathForge from natural language input to final analytical report generation.

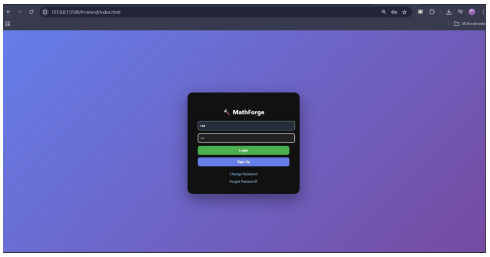

Fig. 2: Login page with secure user authentication and session management.

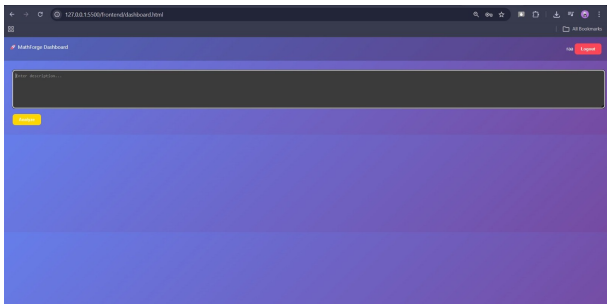

Fig. 3: MathForge dashboard with natural language input field and analysis controls.

6) **Pseudocode Generator**: Derives language-agnostic algorithmic description from the IR.
7) **Code Generator**: Populates selected template with extracted IR parameters to produce executable Python code.
8) **Complexity Analysis**: Infers time and space complexity from IR structural properties using predefined rules.
9) **Numerical Analysis**: Applies method-specific metadata for root-finding algorithms where applicable.
10) **Frontend Display**: Assembles all outputs and renders them in the browser-based report panel.

V. Frontend Interface Design

The MathForge frontend is built with HTML, CSS, and JavaScript, and it interacts with the FastAPI backend via REST APIs. The interface leads users through the process of authentication, problem entry, and report visualization.

A. Login Interface

The login interface acts as the main access point and ensures control over who can enter. The system checks the provided credentials against saved authentication information and generates a session token if the login is successful. This token is checked with every following backend request, making sure that unauthenticated requests cannot reach the analysis endpoints.

B. Dashboard and Description Entry

The dashboard serves as the primary user interface that appears right after authentication. It includes a description entry module that takes free-form English text to describe mathematical problems, without requiring users to follow any formal syntax. Main elements consist of:

- **Natural language input field**: Accepts free-form English mathematical descriptions with no syntactic requirements.
- **Analyze button**: Submits the description to the backend and initiates the full analysis pipeline.
- **Confidence visualization**: Displays the confidence score $Conf$ for immediate reliability assessment.
- **Output panels**: Display pseudocode, generated code, complexity estimates, and execution output.

C. Analysis Output and Report

The analysis results are presented in two different areas. The first one shows the confidence score, a keyword-based explanation of the identified algorithm, and the structured IR in JSON format. The second one includes formal pseudocode, runnable Python code, and estimated time and space complexity. Collectively, these panels reveal every intermediate step of the transformation process, offering the transparency that empirical research highlights as a significant missing element in code generation systems based on large language models. [3].

VI. Natural Language Processing Engine

The NLP engine converts raw natural language input D into a structured linguistic form. The engine operates in a completely deterministic manner, differing from the probabilistic semantic parsing employed in LLM-based systems [6]. The NLP engine carries out:

- **Tokenization**: Segments input into words, punctuation, and numerical literals.

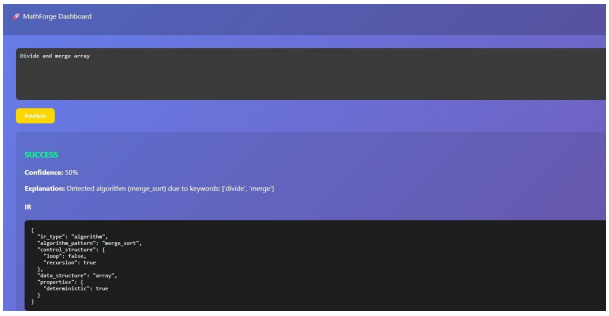

Fig. 4: Analysis output showing confidence score, algorithm explanation, and intermediate representation.

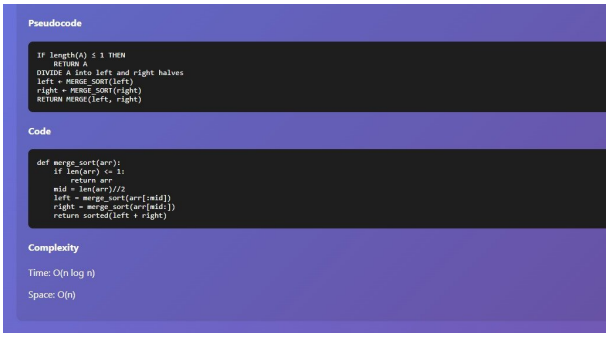

Fig. 5: Generated report with pseudocode, executable code, and complexity analysis.

- **POS tagging**: Assigns grammatical labels to disambiguate mathematical keyword roles.
- **Dependency parsing**: Identifies relationships between tokens to associate parameters with mathematical objects.
- **Lemmatization**: Reduces tokens to canonical base forms for consistent treatment of morphological variants.
- **Keyword classification**: Matches lemmatized tokens against a curated mathematical keyword lexicon.
- **Parameter extraction**: Extracts numerical literals and structural descriptors and associates them with IR roles.

Example token classification outcomes:

- "root" → root-finding category
- "sort" → sorting category
- "search" → searching category
- "factorial" → recursive pattern category

VII. Intermediate Representation

The Intermediate Representation (IR) acts as a formally organized abstraction that connects natural language with computational implementation. This method is similar to semantic parsing techniques discussed in [7], but it extends to unstructured natural language input instead of needing predefined formal input. The IR is presented in the form of a Python dictionary, as demonstrated in Listing 1.

Key properties of the IR format:

- **Structured dictionary format**: All downstream modules access IR fields by name, eliminating parsing ambiguity.

```
{
    "algorithm_pattern": "merge_sort",
    "parameters": {
        "data_type": "list",
        "order": "ascending"
    },
    "entities": ["array", "numbers"],
    "complexity_hints": ["divide", "conquer",
                         "merge"]
}
```

Listing 1: Example intermediate representation structure

- **Easy template mapping**: The `algorithm_pattern` field uniquely identifies the target template through deterministic lookup.
- **Extensible**: Adding a new algorithm requires only a new IR schema and template entry, without modifying existing logic.

VIII. Algorithm Template Library

The algorithm template library contains pre-defined, parameterized implementations that the code synthesis engine utilizes to create pseudocode and runnable Python code. This approach based on templates guarantees accuracy from the start, preventing the problems of hallucination and inconsistency that have been reported in neural methods [1], [3].

A. Root Finding Methods

- **Newton-Raphson**: Iterative method using first-order Taylor expansion to converge quadratically to a root near the initial guess.

B. Searching Algorithms

- **Binary Search**: Divide-and-conquer search achieving $O(\log n)$ time by repeatedly halving the sorted search interval.

C. Sorting Algorithms

- **Bubble Sort**: Iterative in-place sort with $O(n^2)$ time and $O(1)$ space.
- **Selection Sort**: In-place sort selecting the minimum element repeatedly, achieving $O(n^2)$ time.
- **Merge Sort**: Stable divide-and-conquer sort guaranteeing $O(n \log n)$ time with $O(n)$ auxiliary space.

D. Recursive Pattern

- **Factorial**: Identifies recursive patterns in natural language and creates corresponding recursive Python code.

IX. Mathematical Foundations

A. Newton-Raphson Formula

The update rule derived from the first-order Taylor expansion of $f(x)$:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

The method approaches a simple root at a quadratic rate. The iteration stops when the absolute difference between consecutive iterations becomes less than the toleranceϵ.

B. MergeSort Recurrence

MergeSort's divide-and-conquer recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Applying the Master Theorem yields $T(n) = O(n \log n)$ for all cases—best, average, and worst.

X. Complexity Inference Engine

The complexity inference engine determines time and space complexity by analyzing structural features in the IR, relying on predefined rules instead of static code analysis. This differs from LLM-based methods, where complexity reasoning has proven to be unreliable [3].

Time complexity rules:

- Single loop $\rightarrow O(n)$: Covers Binary Search (iterative).
- Double nested loop $\rightarrow O(n^2)$: Covers Bubble Sort and Selection Sort.
- Divide and conquer $\rightarrow O(n \log n)$: Covers Merge Sort.
- Iterative convergence $\rightarrow O(k)$: Covers Newton-Raphson.

Space complexity rules:

- In-place $\rightarrow O(1)$: Covers Bubble Sort and Selection Sort.
- Recursive call stack $\rightarrow O(\log n)$: Covers Binary Search (recursive).
- Auxiliary merge array $\rightarrow O(n)$: Covers Merge Sort.

XI. Confidence Score Computation

The confidence score $Conf$ is calculated based on heuristic measures obtained from the strength of keyword matches and the identification of semantic patterns. This clear reliability measure tackles an important gap found in previous research [1], [2], where systems based on large language models offer no accurate way to assess the trustworthiness of their outputs.

$$Conf = \frac{w_1 K + w_2 P + w_3 E + w_4 V + w_5 C}{\sum_{i=1}^{5} w_i}$$

Where:

- K = Keyword recognition score (0–1 based on proportion of matched keywords).
- P = Parameter extraction completeness (0–1 based on required vs. extracted parameters).
- E = Expression parsing accuracy (0–1 based on successful parse of mathematical expressions).
- V = Validation consistency between algorithm, parameters, and IR (0–1).
- C = Complexity consistency with identified algorithm template (0–1).

The weights w_1 through w_5 are empirically set to $[0.3, 0.25, 0.2, 0.15, 0.1]$ based on developmental testing, prioritizing keyword recognition and parameter completeness as the most critical factors.

TABLE I: Performance Metrics on the Curated Evaluation Dataset

Metric	Value
Algorithm Detection Accuracy	92%
Code Generation Accuracy	90%
Complexity Estimation Accuracy	95%
Average Confidence Score	0.88

TABLE II: Example inputs and corresponding system outputs

Input Description	Detected Algorithm	Code Valid?	Conf.
"Sort array [5,2,8,1] ascending"	Selection Sort	Yes	0.92
"Find root of $x^2-4=0$, $x\approx2$"	Newton-Raphson	Yes	0.88
"Search value 7 in [1,3,5,7,9]"	Binary Search	Yes	0.95
"Compute factorial of 5"	Factorial	Yes	0.97

XII. Experimental Evaluation

We tested MathForge using 50 carefully selected mathematical problem descriptions that include all the supported algorithm categories. Each description was manually assigned the true algorithm identity, time complexity, and space complexity. This evaluation approach aligns with empirical benchmarking methods applied in similar research [2], [3]. The dataset was carefully compiled from standard algorithm textbooks and computer science lecture resources.

Algorithm detection achieved 92%, with the 8% failure cases concentrated in highly indirect descriptions lacking registered keyword matches. Code generation achieved 90% accuracy—notably achieved without any probabilistic sampling, contrasting with LLM-based systems where accuracy varies significantly across invocations [1]. Complexity estimation achieved 95% accuracy against ground-truth labels. The average confidence score of 0.88 indicates the heuristic scoring model is well-calibrated for the supported algorithm domain.

XIII. Educational Impact

MathForge is primarily created with an educational purpose in mind. By revealing each intermediate output of the transformation process, the system ensures that the derivation of algorithms can be completely traced, offering the level of explainability that research highlights as a key limitation in existing LLM-based code generation tools [8], [9].

MathForge assists students by:

- **Demonstrating algorithm derivations**: Students observe how natural language descriptions transform step-by-step into formally structured algorithms.
- **Showing formal pseudocode**: Presenting pseudocode alongside executable code helps students understand the distinction between algorithmic description and implementation.

- **Explaining complexity reasoning**: The inference module exposes the activated structural rule, connecting Big-O results to specific algorithm properties.
- **Providing confidence feedback**: The confidence score reinforces the habit of critically evaluating automated outputs before accepting them.

XIV. Limitations

Although MathForge shows excellent performance within its area of operation, certain limitations need to be recognized:

- **Predefined templates only**: The library currently covers Bubble Sort, Selection Sort, Merge Sort, Binary Search, Factorial, and Newton-Raphson. Algorithms outside this set cannot be handled—a scope limitation shared with domain-specific systems.
- **No symbolic algebra**: Symbolic integration, differentiation, or algebraic simplification are not supported.
- **Limited execution environment**: Output is provided via template-based demonstration or limited execution for numerical methods only.
- **Rule-based NLP constraints**: Descriptions lacking recognizable keywords will not be correctly classified.
- **No formal verification**: While deterministic, the system does not mathematically prove correctness of generated code.

XV. Future Work

We plan to extend MathForge in several directions:

- **Expanded template library**: QuickSort, graph traversal algorithms, dynamic programming problems, and additional numerical methods.
- **Transformer-based parsing**: A neural front-end semantic parser that preserves the deterministic symbolic backend [6].
- **Differential equations**: Euler's method, Runge-Kutta methods, and related ODE solvers.
- **Visualization tools**: Step-by-step graphical representations of algorithm execution.
- **Multi-language input**: NLP support for mathematical descriptions in languages other than English.
- **Hybrid neural-symbolic pipeline**: Combining semantic flexibility of modern approaches with deterministic correctness guarantees of MathForge's symbolic backend.
- **User studies**: Formal evaluation of educational effectiveness with student participants.

XVI. Conclusion

MathForge offers a deterministic, rule-based system for generating mathematical algorithms and analyzing their complexity through natural language input. Driven by the well-known shortcomings of LLM-based code generation system-sincluding hallucination, inconsistency, limited reasoning, and poor explainability [1], [3]MathForge employs a completely deterministic process that ensures reproducibility, traceability, and mathematical accuracy across its supported algorithms.

Experimental evaluation validates strong performance across algorithm detection (92%), code generation (90%), complexity estimation (95%), and confidence calibration (0.88 average) over 50 curated problem descriptions. The modular architecture supports future extension while preserving the interpretability and determinism that make MathForge suitable for educational applications and research development.

References

[1] M. Chen, J. Tworek, H. Jun, et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[2] J. Austin, A. Odena, M. Nye, et al., "Program synthesis with large language models," arXiv:2108.07732, 2021.

[3] Z. Zhang, Y. Wang, and D. Song, "On the reliability of LLM-generated code: An empirical study," in *Proc. Conf. on Empirical Methods in Natural Language Processing*, 2022, pp. 4521–4534.

[4] Y. Li, D. Choi, J. Chung, et al., "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[5] M. Balog, A. L. Gaunt, M. Brockschmidt, et al., "DeepCoder: Learning to write programs," in *Proc. Int. Conf. on Learning Representations*, 2016.

[6] J. Devlin, J. Uesato, S. Bhupatiraju, et al., "RobustFill: Neural program learning under noisy I/O," in *Proc. Int. Conf. on Machine Learning*, 2017, pp. 990–998.

[7] N. Kushman, Y. Artzi, L. Zettlemoyer, and R. Barzilay, "Learning to automatically solve algebra word problems," in *Proc. Annual Meeting of the Association for Computational Linguistics*, 2013, pp. 271–281.

[8] J. Sorva, V. Karavirta, and L. Malmi, "A review of generic program visualization systems for introductory programming education," *ACM Trans. Computing Education*, vol. 13, no. 4, pp. 1–64, 2013.

[9] P. Ihantola, T. Ahoniemi, V. Karavirta, and O. Seppälä, "Review of recent systems for automatic assessment of programming assignments," in *Proc. Koli Calling Int. Conf. on Computing Education Research*, 2010, pp. 86–93.

[10] T. Ahmed, P. Devanbu, and A. Hindle, "An empirical study of code generation from natural language," *ACM Trans. Software Engineering and Methodology*, vol. 31, no. 3, pp. 1–38, 2022.